

The Legend of Zelda: Treasure Hunt Device

COMP5047_Tutorial03_Group5

Xu Liu – 540160639
Yuanxiu He – 540668456
Jinke Wang – 530701686
Xuancheng Zhang – 530590666

1. Project brief

The *Treasure Hunt* project is an interactive device inspired by *The Legend of Zelda: Breath of the Wild* (Nintendo, 2017).
It allows users to experience a real-world treasure-hunting journey through **voice activation**, **BLE proximity sensing**, and **LED light feedback**.
When the player says “Start,” the ESP32 microcontroller activates, connects to the **Gemini API** for semantic intent recognition, and begins scanning BLE signals from the treasure device.
As the user moves closer to the treasure, both the LED brightness and buzzer frequency increase, recreating the excitement of finding a hidden object.

The motivation of this project is to merge fantasy-style gameplay with physical interaction, showing how embedded AI and sensors can bring game-like experiences into everyday environments.
From a technical perspective, it demonstrates how **ESP32**, **Gemini API**, and **BLE communication** can cooperate to form a small, intelligent, and context-aware system.
The BLE module measures proximity through signal strength, a technique commonly applied in context-aware sensing systems (Scheunemann et al., 2019).
The prototype includes a microphone, light sensor, and RGB LED strip, all enclosed in a custom 3D-printed shell inspired by the *Sheikah Slate* from the game.
It fulfills the COMP5047 brief by combining sensing, contextual feedback, and AI-based voice interaction into a single pervasive computing system.

2. Inspiration references

The inspiration for the *Treasure Hunt* project originates from *The Legend of Zelda: Breath of the Wild*, where players use the *Sheikah Slate* to locate hidden treasures in an open world.
We aimed to translate this virtual concept into a **tangible, real-world device**, allowing players to physically search for treasures through light and sound cues.
The 3D-printed enclosure design follows the aesthetic of the *Sheikah Slate*, blending a fantasy theme with modern embedded hardware.
To achieve this, the project draws on existing research in **BLE proximity detection** and **multimodal interaction**.
Scheunemann et al. (2019) demonstrated how Bluetooth Low Energy (BLE) signals can be used to detect human proximity, which directly inspired our distance-sensing mechanism.
Further, recent studies on **voice-gesture integration** show that combining multiple sensory inputs enhances immersion and user engagement (Bai et al., 2023; Li et al., 2023).
This supports our decision to integrate voice control with real-time visual and auditory feedback.

In addition, advances in **AI language and multimodal models** motivated the use of the Gemini API for semantic understanding.
Unlike traditional speech-to-text systems, Gemini allows the device to interpret intent—such as distinguishing “wake” or “stop”—enabling a more natural and intelligent interaction.
Overall, the design merges **game-inspired aesthetics**, **BLE-based sensing**, and **AI-driven intent recognition**, embodying the pervasive-computing goal of embedding intelligence into everyday physical objects.

3. Technical Documentation

DOCUMENTATION BRIEF:
This section provides a detailed explanation of the *Treasure Hunt* system’s design and implementation.
It covers the overall design process, CAD modeling, 3D printing, component preparation, circuit connection, and algorithm logic, as well as the main challenges encountered during development.
Together, these details illustrate how the prototype integrates voice activation, BLE proximity sensing, and adaptive LED/buzzer feedback into a single pervasive-computing system.

3.1 Design Process

The *Treasure Hunt* system was designed around three key goals:

1. Provide an intuitive, game-like treasure-hunting experience.
2. Integrate voice recognition and BLE sensing into a compact IoT device.
3. Deliver clear multimodal feedback using light and sound(Xu et al., 2023).

The interaction flow begins when the user speaks a wake word such as **“Hey! Start.” or “Hey! Finding the treasure.”**
The **ESP32-E** captures the audio, sends it to the **Gemini API** for intent analysis, and—if the intent is valid—switches from *Idle Mode* to *Hunt Mode.*
The device then scans for BLE signals broadcast by the treasure unit.
As the signal strength (RSSI) increases, the RGB LEDs become brighter and the buzzer frequency rises, guiding the player toward the treasure.

3.2 CAD Modeling

Although the team used external 3D models as learning references, all final parts were redesigned and customized to fit the system’s dimensions(Wiberg et al., 2019).
The modeling process was documented using *Fusion 360* for compliance with the course

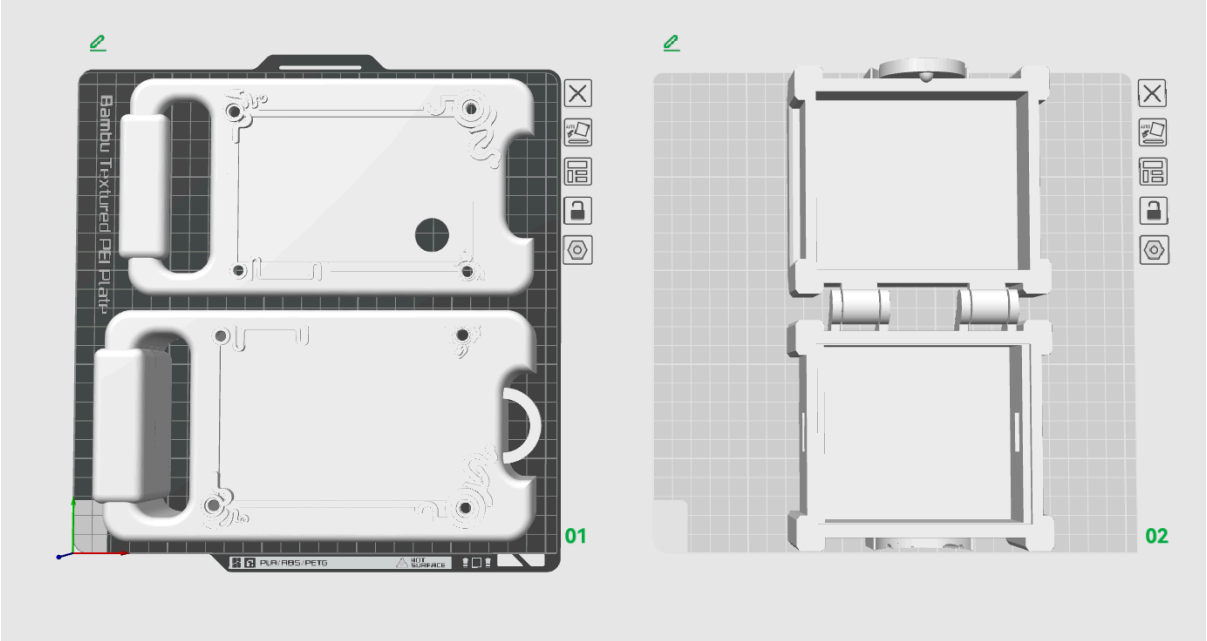
requirement.
The project consists of two physical components:

1. **Treasure Detector (Sheikah Slate)** – the main interactive terminal.
2. **Treasure Box** – the BLE beacon device acting as the hidden target.

For the **Treasure Detector**, the enclosure was divided into an **upper cover** and a **bottom case**.
The upper cover includes openings for multiple indicators:

- a **red LED** for power-on status,
- an **orange LED** for recording and Gemini operation,
- a **common-cathode RGB LED** for treasure-hunting feedback,
- and a **buzzer hole** to output proximity alerts.
  The **bottom case** houses the ESP32 mainboard, battery, wiring, microphone, and LDR light sensor, with additional cutouts for USB debugging and ventilation.

The **Treasure Box** was designed as a small, openable container to hold the ESP32 circuit board and battery.
A side port was added for convenient USB programming and debugging access.
Both components were modeled with simple geometry and compact structure to ensure stability and printability.



3.3 3D Printing



All enclosure parts were printed using **PLA material** with a **0.2 mm layer height** on a **Bambu Lab P1S** printer, as shown in the figures in the Appendix.

The **Treasure Detector**'s upper and lower parts were printed separately, with a **grooved edge** on the top lid that fits precisely onto the bottom case for a firm closure.

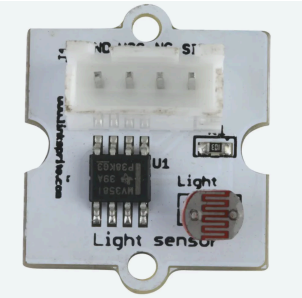
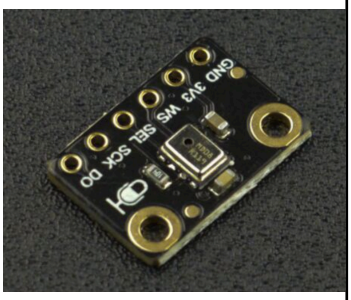
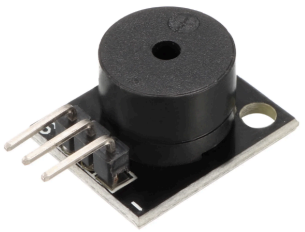
The **Treasure Box** was printed as a **single-piece hinged model**, allowing it to open and close for easy debugging.

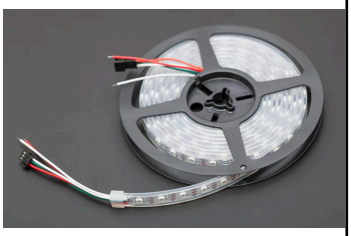
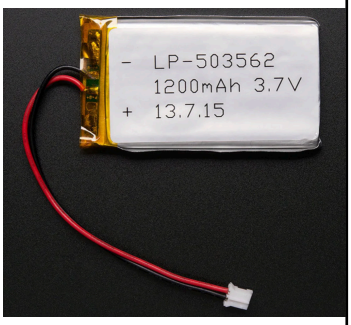
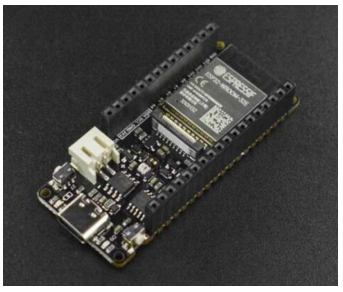
3.4 Component Preparation

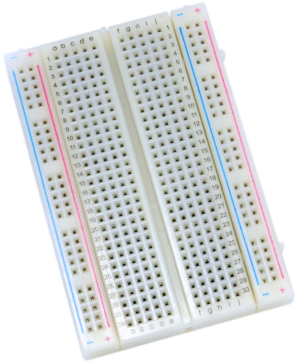
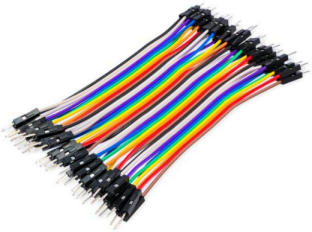
The table below lists all electronic components and mechanical parts used in the *Treasure Hunt* system.

Each item includes its primary function and the quantity used.

The system is composed of two main parts — the **Treasure Detector (Sheikah Slate)** and the **Treasure Box (BLE beacon unit)** both using ESP32 microcontrollers to ensure stable wireless communication and synchronized feedback.

No.	Electronics	Images	Quantity	Used in
1.	**LDR Light Sensor:** Measures ambient brightness for adaptive LED feedback.		1	Treasure Detector
2.	**I2S Microphone (Fermion):** Captures audio input for Gemini API semantic analysis		1	Treasure Detector
3.	**Buzzer:** Provides proximity sound feedback corresponding to BLE signal		1	Treasure Detector
4.	**Red LED:** Indicates Wi-Fi working status		1	Treasure Detector
5.	**Orange LED:** Indicates recording and Gemini API activity		1	Treasure Detector

6.	**RGB LED (Common-Cathode):** Displays treasure-hunt logic and visual feedback based on BLE signal strength		1	Treasure Detector
7.	**220 Ω Resistors:** Current-limiting resistors for two single LEDs and three RGB LED channels; protect ESP32 GPIO pins from overcurrent		5	Treasure Detector
8.	**LED Strip (NeoPixel 3-pin):** For lighting in dark environments		1	Treasure Detector
9.	**Li-ion Battery (3.7 V / 1200 mAh):** Portable power source for both devices		2	Detector & Treasure Box
10.	**ESP32-E Microcontroller:** serves as the control center of the project for low power consumption and miniaturized control.		2	Detector & Treasure Box

11.	**Solderless Breadboard - 400 Tie Points (ZY-60):** Connect Esp32-e and sensor components		1	Treasure Detector
12.	**Dupont Cables:** Connect the main board to the components.		Several	Treasure Detector
13.	**USB Type-C Interface:** For debugging, charging, and firmware upload		1	Detector & Treasure Box
14.	**3D-Printed Enclosure (PLA):** Protects internal circuits; designed as Sheikah Slate and Treasure Box shells		2	Detector & Treasure Box

3.5 Circuit Connection

The following section describes the wiring and pin configuration of the *Treasure Hunt* system.
All sensors, LEDs, and the buzzer are connected to the **ESP32-E microcontroller**, which is **powered directly by a 3.7 V lithium battery**.
A **breadboard** was used to connect the ESP32 with other electronic components and current-limiting resistors during prototyping.

Main Circuit Overview:

- **Power Supply:** 3.7 V Li-ion battery → ESP32 VIN
- **Logic Voltage:** 3.3 V for sensors and LEDs
- **LED Strip:** powered directly from 5 V pin of ESP32
- **Ground:** all modules share a common GND line

Pin Assignments:

Component	ESP32 Pin	Connection Details
Buzzer	GPIO 25	Positive connected to GPIO 25, negative to GND; provides proximity alert sound feedback.
Red LED (220 Ω)	GPIO 12 → LED → GND	Wi-Fi working indicator light.
Orange LED (220 Ω)	GPIO 13 → LED → GND	Indicates recording and Gemini API activity.
RGB LED (Common Cathode)	R → GPIO 21 + 220 Ω G → GPIO 22 + 220 Ω B → GPIO 23 + 220 Ω Common cathode → GND	Displays color feedback based on BLE signal strength.
LED Strip (NeoPixel)	Data → GPIO 17 VCC → 5 V GND → GND	Detected by light sensor, illuminates the treasure hunting terminal in the dark
LDR Light Sensor	Signal → GPIO 36 (A0) VCC → 3.3 V GND → GND	Voltage divider circuit measuring ambient brightness. GPIO 36 corresponds to analog input A0.
I2S Microphone (Fermion)	WS → GPIO 14 SCK → GPIO 16 SD → GPIO 2 SEL → GND GND → GND 3V3 → 3V3	I2S digital audio input for voice capture.

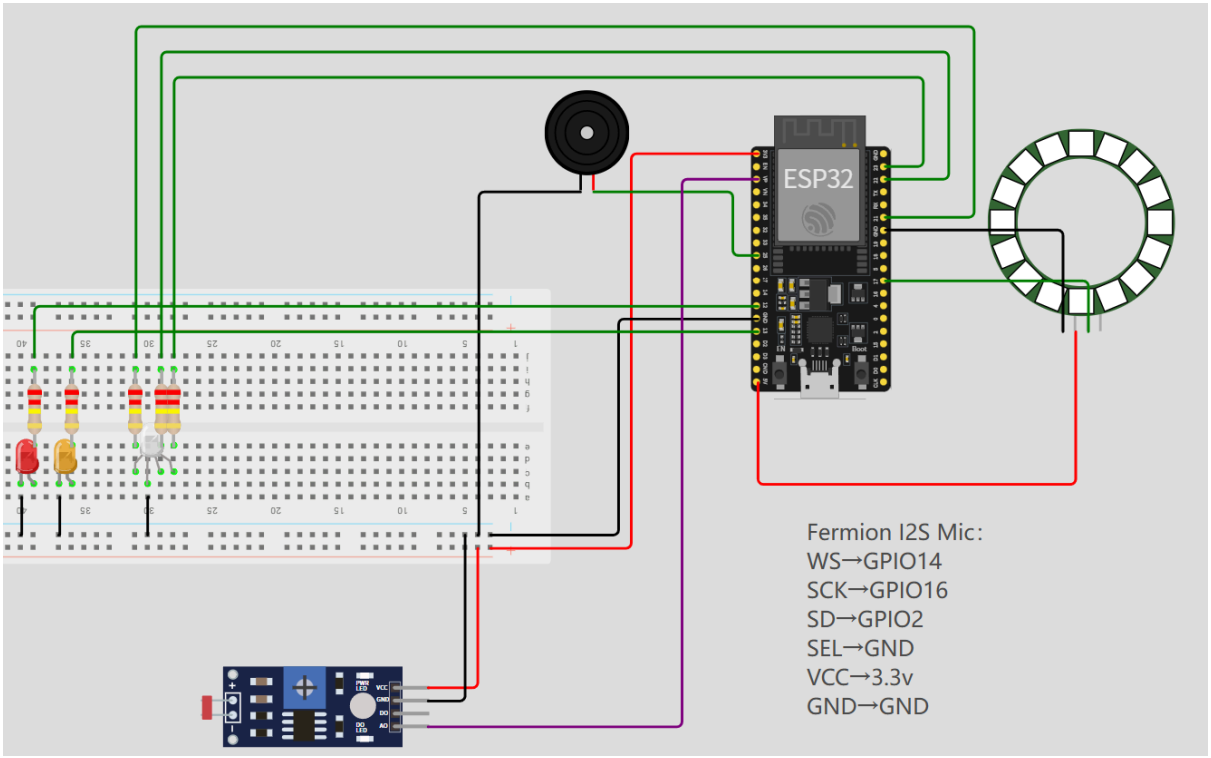
Circuit Notes:

1. **Common-Cathode RGB LED**
   The RGB LED shares a ground pin; each color channel is driven through a 220 Ω resistor to protect the ESP32 GPIO pins.
2. **Single LED Indicators**
   Both red and orange LEDs include 220 Ω resistors and are powered from the 3.3 V logic rail on the breadboard.
3. **LDR Sensor**
   The LDR forms a voltage divider connected to GPIO 36 (A0). The sensor output voltage varies with light intensity, providing analog data for adaptive LED control.
4. **Power Supply and Breadboard Connection**

The ESP32 and all peripherals are powered by a single 3.7V Li-po battery. The breadboard is used to connect the ESP32 pins to LEDs, sensors, and resistors during testing.

5. **BLE Communication and Feedback**
   The detector continuously scans BLE signals broadcast by the treasure box. As the RSSI value increases, LED brightness and buzzer frequency adjust proportionally.

Figure: Circuit Diagram of the Treasure Detector



“ESP32-E Based Treasure Detector Circuit (Wokwi simulation layout)”

Note:

The Fermion I2S microphone module is not available in the simulation library. Its wiring is described textually in the diagram (WS→GPIO14, SCK→GPIO16, SD→GPIO2, VCC→3.3 V, GND→GND).

4. Algorithms And Code

4.1 System Algorithm Overview

The Treasure Hunt system integrates audio, BLE, and light sensors under a state-based control algorithm.

When the device is powered on, it stays in Idle mode with the microphone continuously monitoring background sound. Once the RMS amplitude exceeds a preset threshold, the system records a short audio clip and sends it to the Gemini API for intent

classification. If Gemini returns “WAKE”, the device enters Scan mode, where it continuously scans for the BLE signal emitted by the treasure node (TREASURE_05). The RSSI value is processed through a median filter and mapped to different LED colors and buzzer frequencies to indicate proximity. When the signal becomes strong enough, the device plays a success melody and returns to Idle mode. This state-driven architecture ensures low power usage, responsive interaction, and intuitive multimodal feedback through sound and light.

4.2 Main Code Implementation

Treasure Detector:

(1) setupI2SMic()

```
/* ========= I2S Microphone Setup ========= */
void setupI2SMic(){
  i2s_config_t cfg={
    .mode=(i2s_mode_t)(I2S_MODE_MASTER|I2S_MODE_RX),
    .sample_rate=SAMPLE_RATE,
    .bits_per_sample=I2S_BITS_PER_SAMPLE_16BIT,
    .channel_format=I2S_CHANNEL_FMT_ONLY_LEFT,
    .communication_format=(i2s_comm_format_t)I2S_COMM_FORMAT_STAND_I2S,
    .intr_alloc_flags=ESP_INTR_FLAG_LEVEL1,
    .dma_buf_count=8,
    .dma_buf_len=64,
    .use_apll=false
  };
  i2s_pin_config_t pin={I2S_PIN_NO_CHANGE,I2S_SCK,I2S_WS,I2S_PIN_NO_CHANGE,I2S_SD};
  i2s_driver_install(I2S_PORT,&cfg,0,NULL);
  i2s_set_pin(I2S_PORT,&pin);
  Serial.println("✅ I2S Microphone Initialized");
}
```

(2) getMicRMS()

```
/* ========= Compute RMS (Root Mean Square) of Audio Samples ========= */
double getMicRMS(){
  int16_t s[READ_LEN];
  size_t br;
  i2s_read(I2S_PORT,(char*)s,READ_LEN*sizeof(int16_t),&br,portMAX_DELAY);
  int n=br/sizeof(int16_t);
  if(n<=0)return 0;
  double amp=20.0,sum=0;
  for(int i=0;i<n;i++){
    int v=(int)(s[i]*amp);
    if(v>32767)v=32767;
    if(v<-32768)v=-32768;
    s[i]=(int16_t)v;
    sum+=(double)v*v;
  }
  return sqrt(sum/n);
}
```

(3) connectWiFi()

```cpp
/* ========= Wi-Fi Connection ========= */
bool connectWiFi(){
  WiFi.begin(ssid,password);
  Serial.print("🌐 Connecting WiFi");
  int r=0;
  while(WiFi.status()!=WL_CONNECTED&&r<25){
    delay(500);
    Serial.print(".");
    r++;
  }
  if(WiFi.status()==WL_CONNECTED){
    Serial.printf("\n✅ WiFi Connected: %s\n",WiFi.localIP().toString().c_str());
    return true;
  }
  Serial.println("\n❌ WiFi Failed");
  return false;
}
```

(4) detectIntentFromAudio() — Gemini

```cpp
/* ========= Gemini API Call: Audio Intent Detection ========= */
String detectIntentFromAudio(const String& b64){
  orangeMode = ORANGE_BLINK;
  Serial.println("🟠 Gemini analyzing... (orange LED blinking)");
  WiFiClientSecure cli;cli.setInsecure();HTTPClient https;
  String url="https://generativelanguage.googleapis.com/v1beta/models/"
             "gemini-2.5-flash-preview-05-20:generateContent?key="+GEMINI_API_KEY;
  if(!https.begin(cli,url)){
    Serial.println("❌ HTTPS begin fail");
    orangeMode=ORANGE_OFF;
    return"NONE";
  }

  // Construct request payload
  DynamicJsonDocument doc(8192);
  JsonArray contents=doc.createNestedArray("contents");
  JsonObject content=contents.createNestedObject();
  JsonArray parts=content.createNestedArray("parts");
  String prompt=R"(
You are an audio intent detector for a treasure-hunting device.
Input: a short speech audio clip (English or Chinese).
Goal: decide whether the user wants to START/CONTINUE searching (WAKE),
STOP/PAUSE/END/mute the search (STOP), or it's irrelevant (NONE).

Rules:
- STOP has priority when ambiguous (e.g., 'be quiet', 'mute it', 'enough').
- Treat questions like 'where is my treasure?' / '我们开始吧' / '开始找宝藏' as WAKE.
- Treat phrases like 'we found it', 'stop searching', '别找了', '关闭它', '停止搜索' as STOP.
- If uncertain, answer NONE.

Respond ONLY with one of the following tokens (case-insensitive): WAKE, STOP, NONE.
)";
  parts.createNestedObject()["text"]=prompt;
  JsonObject inline_data=parts.createNestedObject()["inline_data"].to<JsonObject>();
  inline_data["mime_type"]="audio/l16;rate=16000";
  inline_data["data"]=b64;

  // Send POST request
  String payload;
  serializeJson(doc,payload);
  https.addHeader("Content-Type","application/json");
  https.setTimeout(40000);
  Serial.printf("📡 Sending JSON (%.1f KB)...\n",payload.length()/1024.0);
  int code=https.POST(payload);

  if(code<=0){
    Serial.printf("❌ POST fail: %s\n",https.errorToString(code).c_str());
```

```
268     if(code<=0){
269       Serial.printf("❌ POST fail: %s\n",https.errorToString(code).c_str());
270       https.end();
271       orangeMode=ORANGE_OFF;
272       return"NONE";
273     }
274 
275     String resp=https.getString();
276     https.end();
277     orangeMode=ORANGE_OFF;
278     Serial.println("🔵 Gemini analysis complete.");
279 
280     DynamicJsonDocument rdoc(8192);
281     if(deserializeJson(rdoc,resp)){
282       Serial.println("❌ JSON parse error");
283       return"NONE";
284     }
285 
286     String text=rdoc["candidates"][0]["content"]["parts"][0]["text"]|"NONE";
287     text.toUpperCase();
288     text.trim();
289     Serial.printf("🤖 Gemini raw: '%s'\n",text.c_str());
290     if(text=="STOP")return"STOP";
291     if(text=="WAKE")return"WAKE";
292     return"NONE";
293   }
```

(5) loop()

The following code represents the main control loop of the Treasure Hunt system.

It continuously monitors light intensity, microphone RMS, and BLE RSSI values to determine the device's behavior.

The algorithm implements a simple finite state machine with two modes:

- IDLE mode: waiting for voice commands.
- SCAN mode: actively scanning BLE signals and giving real-time audio-visual feedback.

```
/* ========= Main Loop ========= */
void loop(){
  // Ambient light control for LED strip
  int ldr=analogRead(LDR_PIN);
  static bool lamp=false;
  if(!lamp&&ldr<LDR_ON_TH)lamp=true;
  if(lamp&&ldr>LDR_OFF_TH)lamp=false;
  if(lamp){strip.fill(strip.Color(255,255,255));}
  else{strip.clear();}
  strip.show();

  // Display periodic sensor values
  static unsigned long last=0,lastTrig=0;
  if(millis()-last>1000){
    last=millis();
    lastRMS=getMicRMS();
    int rssi_for_print=(latestRSSI==-127)?-127:median5(latestRSSI);
    Serial.printf("🌞 LDR=%d | 🎙 RMS=%.1f | RSSI=%d | Mode=%s\n",
      ldr,lastRMS,rssi_for_print,mode==IDLE?"IDLE":"SCAN");
  }

  // Re-enable microphone after cooldown
  if(!micEnabled&&allowMicAutoResume&&millis()>=micResumeTime){
    micEnabled=true;
    Serial.println("🎙 Mic re-enabled after cooldown.");
  }

  // Voice trigger logic (Gemini speech intent detection)
  if(micEnabled&&(millis()-lastTrig>VOICE_COOLDOWN_MS)&&lastRMS>RMS_TRIGGER_THRESHOLD){
    lastTrig=millis();
    bool wasScanning=(mode==SCAN);
    if(wasScanning)pScan->stop();

    int16_t*recBuf=(int16_t*)malloc(g_recBytes);
    if(!recBuf){Serial.println("❌ Memory allocation failed");return;}
    Serial.println("🎤 Voice trigger detected!");

    // Record short audio clip
    orangeMode = ORANGE_SOLID;
    size_t br = 0;
    unsigned long recStart = millis();
    while (millis() - recStart < 2000) {
      size_t bytesRead = 0;
      i2s_read(I2S_PORT, (uint8_t*)recBuf, READ_LEN * sizeof(int16_t), &bytesRead, portMAX_DELAY);
      br += bytesRead;
    }
    orangeMode = ORANGE_OFF;
```

```
385       // Compute amplitude and filter empty audio
386       int16_t maxAmp=0;
387       for(size_t i=0;i<br/sizeof(int16_t);i++)
388         if(abs(recBuf[i])>maxAmp)maxAmp=abs(recBuf[i]);
389       if(maxAmp<300){Serial.println("🔴 Audio too quiet, skipped.");free(recBuf);return;}
390
391       // Send to Gemini API
392       String b64=Base64::encode((uint8_t*)recBuf,br);
393       free(recBuf);
394       String intent=detectIntentFromAudio(b64);
395       Serial.printf("🔍 Intent detected: %s\n",intent.c_str());
396
397       if(intent=="WAKE"){
398         mode=SCAN;
399         micEnabled=false;
400         allowMicAutoResume=false;
401         scanStartTime=millis();
402         if(pScan->isScanning())pScan->stop();
403         delay(100);
404         pScan->start(0,nullptr);
405         Serial.println("▶️ Enter SCAN mode");
406       }
407       else if(intent=="STOP"){
408         mode=IDLE;
409         Serial.println("⏸ Enter IDLE mode");
410       }
411       if(wasScanning&&intent!="WAKE"&&intent!="STOP"){
412         pScan->start(0,nullptr);
413       }
414     }
415
416     // Idle mode (LEDs off)
417     if(mode==IDLE){setRGB(0,0,0);pScan->stop();delay(60);return;}
418
419     // SCAN mode: distance-based feedback
420     if(mode==SCAN){
421       if(!pScan->isScanning()){
422         pScan->start(0,nullptr);
423         Serial.println("🔁 BLE scan running...");
424       }
425
426       // No BLE signal → slow green heartbeat flash
427       if(latestRSSI == -127 || (millis() - lastSeenMs) > STALE_MS) {
428         static unsigned long blinkT = 0;
429         if (millis() - blinkT > 2000) {
```

```
      static unsigned long blinkT = 0;
      if (millis() - blinkT > 2000) {
        blinkT = millis();
        setRGB(0, 60, 0);
        delay(300);
        setRGB(0, 20, 0);
      }
      delay(60);
      return;
    }

    int rssi=median5(latestRSSI);
    if(rssi<=-72)setRGB(0,220,0);
    else if(rssi<-62)setRGB(0,0,255);
    else setRGB(255,0,0);

    static uint8_t q=0;
    static unsigned long qs=0,refr=0;
    if(millis()<refr){delay(20);return;}

    // Trigger melody when treasure signal quality is high
    if(rssi>=QL_LOW&&rssi<=QL_HIGH){
      if(q==0||millis()-qs>QL_WINDOW_MS){q=1;qs=millis();}
      else{q++;if(q>=QL_HITS){playFoundTone();q=0;refr=millis()+500;}}
    }else if(q>0&&millis()-qs>QL_WINDOW_MS)q=0;

    // Final found condition
    if(rssi>=TH_FOUND&&(millis()-scanStartTime>3000)){
      playFoundTone();
      pScan->stop();
      mode=IDLE;
      micEnabled=false;
      allowMicAutoResume=true;
      micResumeTime=millis()+5000;
      Serial.println("🏁 Treasure found! Mic will re-enable in 5s.");
      refr=millis()+500;
      return;
    }

    // Distance-based buzzer feedback
    if(rssi<TH_SILENT)delay(60);
    else if(rssi<TH_SLOW){beep(30);delay(900);}
    else if(rssi<TH_MED){beep(40);delay(650);}
    else if(rssi<TH_FAST){beep(45);delay(180);beep(45);delay(280);}
    else if(rssi<TH_VFAST){for(int i=0;i<3;i++){beep(55);delay(140);}}
    else{unsigned long t=millis();while(millis()-t<450){beep(35);delay(75);}}
  }
}
```

This loop integrates multimodal sensing, Gemini intent recognition, and BLE-based proximity feedback, forming the core logic of the Treasure Hunt system.

(6) BLE Scanning Routine

```
/* ========== BLE Advertisement Callback ========== */
class AdvCB:public NimBLEAdvertisedDeviceCallbacks{
  void onResult(NimBLEAdvertisedDevice*adv)override{
    if(!adv->haveName())return;
    if(adv->getName()==TARGET_NAME){
      latestRSSI=adv->getRSSI();
      lastSeenMs=millis();
    }
  }
};
```

Treasure Box:

(1) BLE Beacon Transmitter

```
void setup() {
  // Initialize BLE device with the name "TREASURE_05"
  BLEDevice::init("TREASURE_05");

  // Set maximum transmit power (+9 dBm)
  BLEDevice::setPower(ESP_PWR_LVL_P9);

  // Create BLE server and advertising instance
  BLEServer *pServer = BLEDevice::createServer();
  BLEAdvertising *pAdvertising = pServer->getAdvertising();

  // Prepare the advertisement data packet
  BLEAdvertisementData adv;
  adv.setName("TREASURE_05");
  adv.setFlags(ESP_BLE_ADV_FLAG_GEN_DISC | ESP_BLE_ADV_FLAG_BREDR_NOT_SPT);

  // Apply advertisement data and start broadcasting
  pAdvertising->setAdvertisementData(adv);
  pAdvertising->start();

  // Initialize serial monitor for debugging
  Serial.begin(115200);
  Serial.println("Treasure broadcasting...");
}

void loop() {
  // Periodically print status message for verification
  Serial.println("Still broadcasting...");
  delay(2000);   // Print every 2 seconds
}
```

4.3 Brief Explanation

The code implements an interactive treasure-hunting system using an ESP32 microcontroller integrated with multiple sensors and AI-based intent detection.
It continuously monitors ambient light, sound, and Bluetooth signal strength to control LEDs, a buzzer, and system states.

- **Voice Activation (Gemini AI):**
  The microphone captures short audio clips when the sound intensity exceeds a threshold.
  The clip is encoded and sent to Google Gemini via HTTPS for intent classification — *WAKE*, *STOP*, or *NONE*.
  Depending on the intent, the system either starts scanning for the treasure or returns to idle mode.
- **BLE Proximity Feedback:**
  During the scan mode, the ESP32 measures RSSI values from the treasure's BLE beacon(Kim & Lee, 2023).
  The RGB LED changes color based on signal strength, and the buzzer produces faster tones as the device gets closer to the treasure.
- **Light Sensor & LED Strip:**
  A photoresistor automatically turns on the LED strip in low-light conditions to illuminate the device enclosure.
- **System Feedback & Debugging:**
  Serial logs show LDR readings, microphone RMS, BLE RSSI, and mode transitions for real-time observation and performance evaluation.

5. Assembling the parts

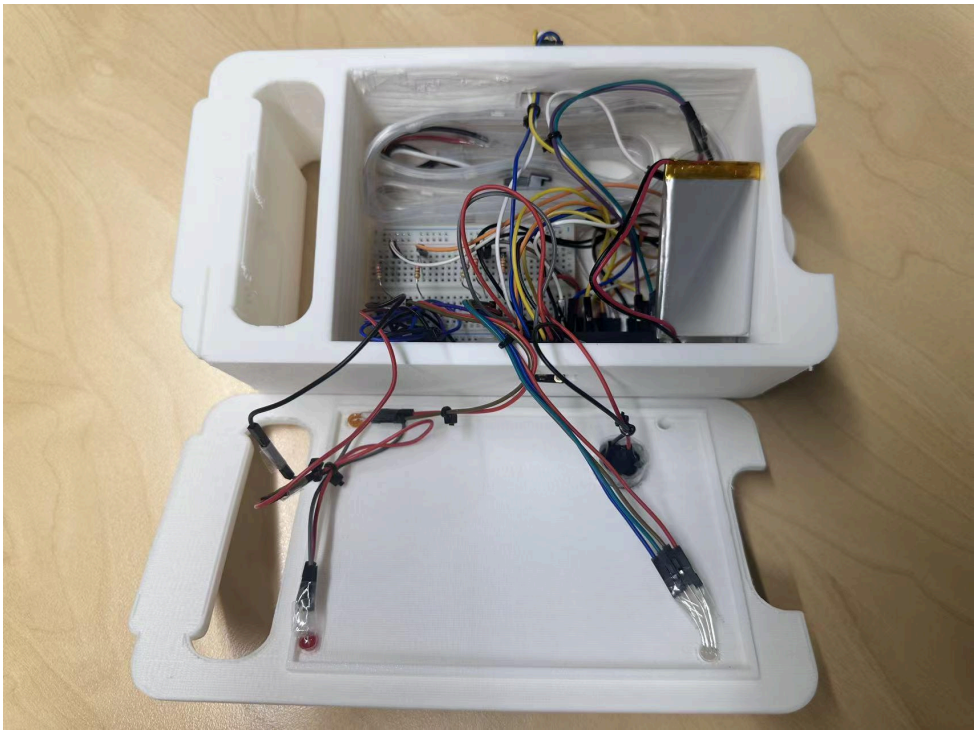
5.1 Assembly Overview

The prototype consists of two separate units: the **Treasure Detector(Sheikah Slate)** and the **Treasure Box(Beacon Unit)**.
Both are powered by Li-ion batteries and communicate wirelessly via BLE.
The ESP32 on the terminal performs sensing, Gemini-based voice processing, and feedback control, while the ESP32 on the chest simply broadcasts a BLE signal to be detected.

5.2 Treasure Detector Assembly

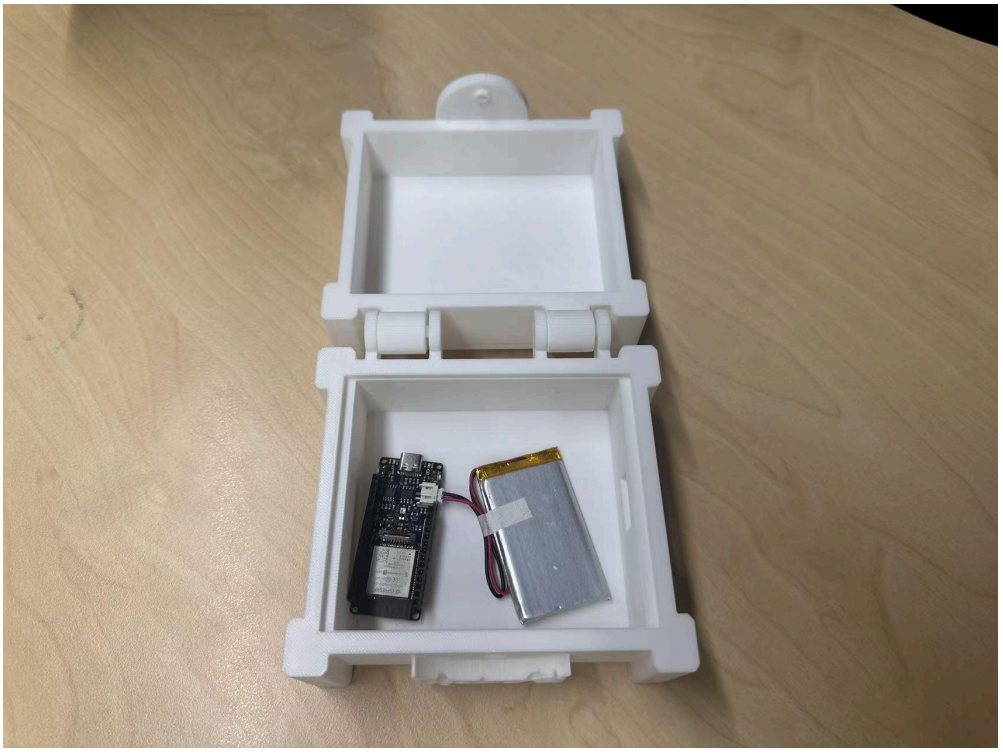
All electronic components—including the ESP32-E board, microphone, LDR sensor, RGB LED, indicator LEDs, buzzer, and LED strip—were wired on a small breadboard.
After verifying circuit connections and functionality on the serial monitor, the assembly was placed inside the 3D-printed "Sheikah Slate" enclosure.
The top cover provides openings for the LEDs, buzzer, and sensors.

Once all components were secured, the cover was closed, completing the terminal unit.



5.3 Treasure Box Assembly

The treasure unit contains an ESP32 board pre-programmed with the BLE advertising sketch (TREASURE_05).
The board and its Li-ion battery are placed directly inside the 3D-printed chest enclosure.
A small opening allows USB access for charging or firmware upload.
Once closed, the chest continuously broadcasts its signal and is ready for testing.



5.4 Final Setup

After all wiring was verified, both units were powered on.
The terminal connects to Wi-Fi (red LED ON) and starts in Idle Mode.
When the user speaks a command such as “Start,” the system enters Scan Mode, begins scanning for BLE signals, and provides both visual (RGB LED) and audio (buzzer) feedback as the treasure is approached.
Once the treasure is found, the system confirms success through rapid red LED flashing and a distinct buzzer tone.

6. Challenges Encountered

Coding:

1. The initial design of the Gemini interaction and voice-recording logic was implemented on a computer using Python.
   The ESP32 simply acted as a data source while the API connection ran on the PC.
   Later, the tutor clarified that the final prototype must be completely independent — powered by its own battery and not connected to a computer.
   Considerable time was therefore spent migrating all Gemini communication logic onto the ESP32-E board.

2. During this migration, serious issues arose when sending recorded audio to Gemini because of the ESP32’s limited memory capacity(Lee et al., 2025).
   The board frequently timed out while transmitting the encoded audio data, and several rounds of code optimisation and buffer reduction were required before the API responded reliably.

Materials:

1. Additional components such as PH-2-connector Li-ion batteries and LED strips were ordered later in the semester.
   Their delivery delays slowed the integration and debugging progress, postponing final testing.

Design and 3D printing

1. The Treasure Detector enclosure was printed once without adequate supports, causing slight surface damage on the upper shell, though functionality was not affected.
2. The printing session originally booked on campus was insufficient for the ten-hour build time. Because the tutor had to close the lab and the printer slots were fully booked afterwards, the final enclosures for both the Treasure Detector and the Treasure Box were printed externally.

3. During design, significant effort was spent determining enclosure dimensions, hole alignment, and the number and size of sensor openings to ensure that all modules could fit properly.

Assembling:

1. The buzzer opening on the Treasure Detector lid was slightly oversized.
   The issue was solved by wrapping the buzzer edge with tape to increase its thickness for a tighter fit.
2. The light sensor was originally placed inside the case, exposing only its photoresistor to ambient light.
   However, the LED strip's illumination was reflected onto the sensor, causing a feedback loop where the strip turned on in the dark, triggered the sensor, and immediately turned itself off.
   Relocating the light sensor outside the enclosure and mounting the LED strip inside eliminated this problem.
3. Several jumper wires were too short to reach between components; this was resolved by connecting male-to-female Dupont wires in pairs to extend their length.

7. User Manual

7.1 System Overview

The Treasure Hunt system consists of two parts:

- **Treasure Detector** (the handheld seeker terminal)
- **Treasure Box** (the BLE beacon target)

The detector listens for BLE signals broadcast by the treasure box and provides **visual (RGB LED + LED strip)** and **audio (buzzer)** feedback to help the player locate the treasure.

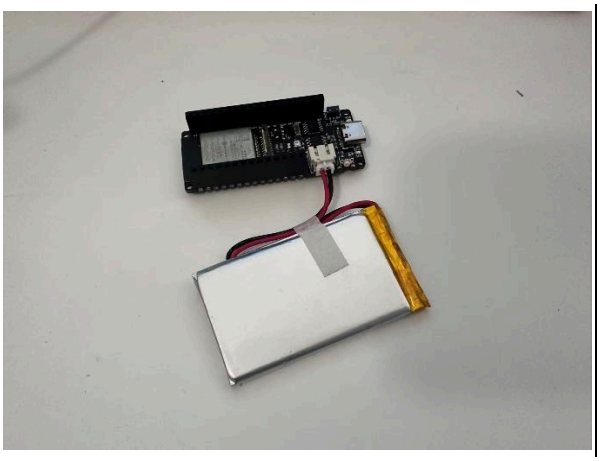
It also features **voice interaction** powered by Gemini API, allowing the user to start or stop searching using spoken commands.

7.2 Power Supply

Both units are powered by **3.7 V Li-ion batteries** with PH-2 connectors.

- **Treasure Detector:** 3.7 V Li-ion battery (or USB 5 V for debugging)
- **Treasure Box:** same 3.7 V Li-ion battery

Also, charging can be through the ESP32 USB-C port.





7.3 How to Operate

1 Power On

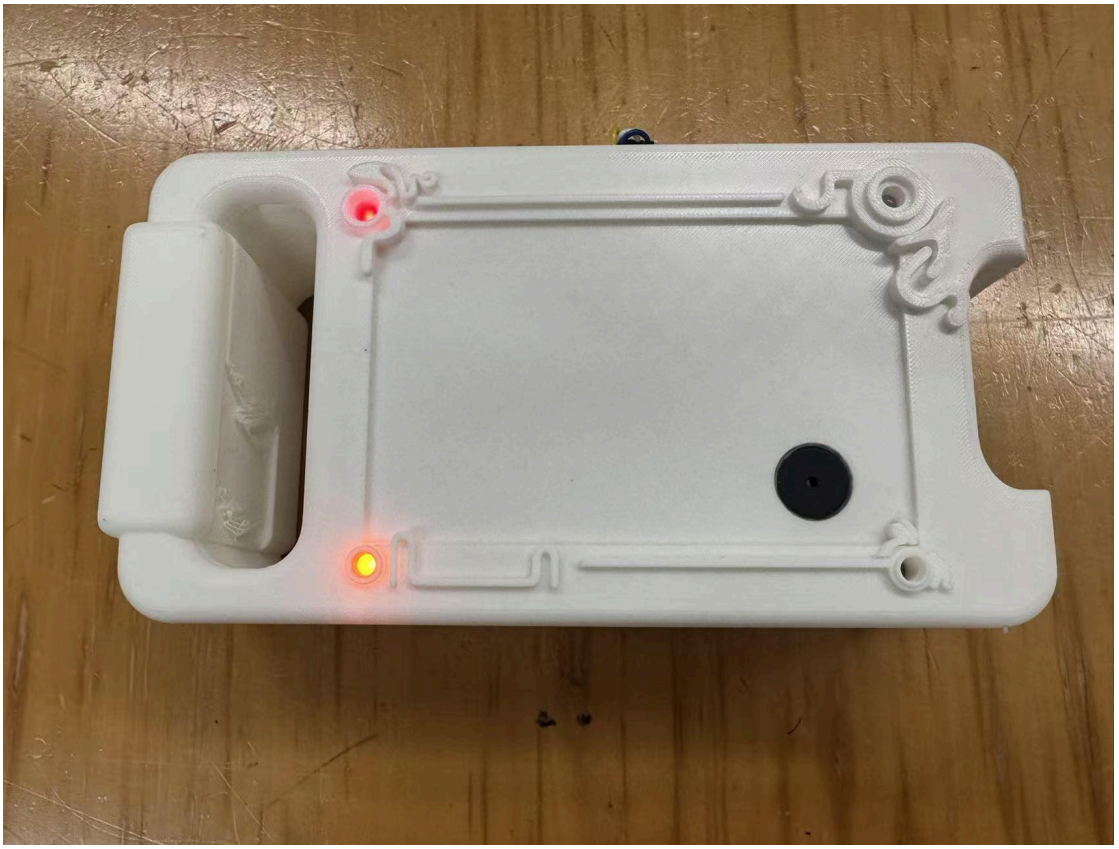
- Turn on both the Treasure Detector and the Treasure Box.
- The red LED on the detector indicates Wi-Fi status:
  - ON → Wi-Fi connected successfully.
  - OFF → Connection failed (the device will still function offline).



2 Voice Commands

- Speak clearly near the microphone (for example: “Hey! start”, “Hey! let’s find it”).
- The device records a short audio clip and sends it to Gemini for intent recognition.

    - While Gemini is analyzing, the **orange LED blinks**.
    - When analysis is complete, the orange LED turns **off**.
- If Gemini classifies the command as WAKE, the system enters Treasure Hunt (Scan) Mode and begins searching for the BLE signal.
- If Gemini classifies it as NONE, the device ignores the input and remains in Idle Mode.



To ensure a smooth treasure-hunting experience, the microphone is automatically disabled after activation.
This prevents ambient sounds or random speech during the hunt from accidentally triggering additional voice detection, allowing the user to focus entirely on finding the treasure.

3 Treasure Hunt Mode
When the Treasure Detector receives BLE signals from the Treasure Box, it provides both light and sound feedback based on the signal strength (RSSI):

RSSI Range	**RGB LED Color**	**Buzzer Pattern**	**Meaning**
–70 to –60 dB	Green	Slow beep	Far from treasure
–60 to –50 dB	Blue	Medium beep	Getting closer
> –50 dB	Red	Fast / continuous beep	Treasure nearby

(Change parameters according to site conditions)



4 Automatic Lighting

The LED strip is controlled by the light sensor (LDR).

It automatically turns on when the environment is dark and off in bright conditions.



7.4 System Reset & Testing

- Press the EN button on ESP32 to reboot.
- Speak near the microphone — if the serial monitor prints “Voice trigger detected!”, the mic works correctly.
- Turn off the Treasure Box — the detector’s LED will blink slowly in green, meaning no BLE signal.

7.5 Safety & Maintenance

- Do not short-circuit or overcharge the battery.
- Avoid prolonged exposure to direct sunlight.
- If the LED strip flickers, check that the light sensor is not illuminated by it.
- Disconnect the battery for long-term storage.

7.6 Troubleshooting

Problems	**Possible Cause**	**Solutions**
Wi-Fi LED is OFF	Wrong SSID / password or no Wi-Fi available	Update the Wi-Fi credentials in code and re-upload if Wi-Fi is

		required.
Orange LED keeps blinking	Gemini is timing out or network is unstable	Check Wi-Fi signal and confirm the Gemini API key is valid.
No buzzer sound during hunting	Buzzer not wired correctly / wrong pin	Verify the buzzer is connected to GPIO 25 and ground.
LED strip flickers on/off repeatedly	Light from the LED strip is hitting the light sensor (LDR)	Make sure the LDR is placed outside the enclosure and the LED strip is inside.
No BLE feedback / always “far away”	Treasure Box is off or out of range	Power on the Treasure Box or move it closer to the Detector.
Saying words does nothing	Gemini classified the speech as NONE (not WAKE)	Speak louder and closer to the microphone, and use a full activation phrase like “Hey, start”.

8. References

Bai, A., Shahid, I., Takawale, H., & Roy, N. (2023). WhisperWand: Simultaneous voice and gesture tracking interface. arXiv preprint arXiv:2301.10314. https://arxiv.org/abs/2301.10314

Kim, K., & Lee, J. (2023). Adaptive Scheme of Denoising Autoencoder for Estimating Indoor Localization Based on RSSI Analytics in BLE Environment. *Sensors (Basel, Switzerland)*, *23*(12), 5544. https://doi.org/10.3390/s23125544

Lee, S., Kang, W., Bertogna, M., Chwa, H. S., & Lee, J. (2025). Timing guarantees for inference of AI models in embedded systems. *Real-Time Systems*, *61*(2), 259–267. https://doi.org/10.1007/s11241-025-09445-9

Li, Z., Liang, C., Wang, Y., Qin, Y., Yu, C., Yan, Y., Fan, M., & Shi, Y. (2023). Enabling voice-accompanying hand-to-face gesture recognition with cross-device sensing. arXiv preprint arXiv:2303.10441. https://arxiv.org/abs/2303.10441

Scheunemann, M. M., Dautenhahn, K., Salem, M., & Robins, B. (2019). Utilizing Bluetooth Low Energy to recognize proximity, touch and humans. arXiv preprint arXiv:1904.03475. https://arxiv.org/abs/1904.03475

Wiberg, A., Persson, J., & Ölvander, J. (2019). Design for additive manufacturing – a review of available design methods and software. *Rapid Prototyping Journal*, *25*(6), 1080–1094. https://doi.org/10.1108/RPJ-10-2018-0262

Xu, H., Zhang, H., & Lin, T.-E. (2023). *Intent recognition for human-machine interactions* (1st ed. 2023.). Springer Nature Singapore. https://doi.org/10.1007/978-981-99-3885-8

9. Appendix

3D Printing Reference:







